Java 入门笔记

第十三课 · 封装、构造方法与参数传递

学习目标	上节课用 `private` 让外部无法直接修改 `speed`。本课把它发展成完整的封装：私有属性加 getter/setter；再用构造方法在创建对象时一次性完成初始化；最后弄清 Java 方法调用时参数到底是怎样传递的。

目录

本课目标 2

一、复习：方法重载 2

二、访问权限补充 2

三、封装 3

四、构造方法 5

五、Java 参数传递 7

六、课上遇到的问题 9

常见错误 9

本课小结 9

自测题 10

整理日期：2026 年 11 月 6 日

本课目标

- 能判断两个方法是否构成重载。
- 分清 `private`、默认、`protected` 在同包和跨包时的访问范围。
- 会用 `private` 加 getter/setter 实现封装，会用 IDEA 自动生成。
- 掌握构造方法的三条规则，以及“默认构造方法”的陷阱。
- 理解 Java 只有值传递，能说出基本类型、对象、数组作为参数时的区别。

一、复习：方法重载

两同一不同：同一个类、方法名相同，参数列表不同。 参数列表不同指参数的个数、类型或顺序至少有一项不同。

只有这里不同	能否构成重载
参数个数、类型或顺序	能
返回值类型	不能，编译报错
参数名	不能
访问修饰符	不能

```
void show(int a) { }
void show(int a, int b) { }      // 个数不同 ✓
void show(double a) { }          // 类型不同 ✓
void show(int a, String s) { }
void show(String s, int a) { }   // 顺序不同 ✓
// int show(int a) { }           ✗ 只有返回值不同，编译报错
```

调用时，编译器根据实参选择最匹配的方法；没有完全匹配时，尝试自动类型提升：

```
show(10);     // show(int)
show(10.5);   // show(double)
show('A');    // 没有 show(char)，char 提升为 int，调用 show(int)
```

课堂代码 `Car` 中的 `fly()` 与 `fly(String color)` 就构成重载。重载只看参数，两个方法的访问修饰符不同（如一个 `private`、一个默认）也不影响。

二、访问权限补充

修饰符	同一个类	同一个包	不同包的子类	不同包的其他类
`private`	✓	✗	✗	✗
默认	✓	✓	✗	✗
`protected`	✓	✓	✓	✗
`public`	✓	✓	✓	✓

1. 同包可以互相访问的是“非 **private**”成员

默认、`protected`、`public` 成员在同一个包内都能直接访问；`private` 只有本类自己能访问，同包也不行。

本课 `Car` 把 `speed` 设为 `private`，但 `main` 写在 `Car` 类内部，所以 `a.speed = 180;` 合法。上节课同样的语句写在另一个类 `Test` 里，就编译报错。

```
class Car {
    private int speed;

    public static void main(String[] args) {
        Car a = new Car();
        a.speed = 180;      // ✓ 在 Car 自己的类里
    }
}

class Test {
    void test(Car car) {
        car.speed = 180;    // ✗ speed has private access in Car
    }
}
```

2. 不同包的子类通过继承访问 **protected**

```
package class14;
import class13.Car;               // Car 必须是 public 类

public class SportsCar extends Car {
    void test() {
        color = "red";            // ✓ 继承来的 protected 成员
        Car c = new Car();
        c.color = "blue";         // ✗ 不是自己继承的那一份
    }
}
```

不同包的子类只能访问自己继承来的 `protected` 成员；通过父类对象去访问不行。跨包使用时，父类本身还要声明为 `public`，否则别的包看不到这个类。

三、封装

封装：把属性设为 `private` 隐藏起来，只通过类提供的公共方法访问。 外部只能按类规定的方式使用对象。

1. 为什么要封装

不封装时，外部可以写入任何值，类无法阻止不合理的数据：

```
car.speed = -100;     // 速度为负，不合理，但能通过编译
```

2. 标准写法：private 属性 + getter/setter

```
public class Car {
    private int speed;

    public void setSpeed(int speed) {          // setter：设置值，可以做校验
        if (speed >= 0 && speed <= 300) {
            this.speed = speed;
        } else {
            System.out.println("速度不合法");
        }
    }

    public int getSpeed() {                    // getter：获取值
        return speed;
    }
}

car.setSpeed(-100);   // 输出“速度不合法”，speed 仍为 0
car.setSpeed(120);    // speed 变为 120
```

规范	例子
setter 命名	`setXxx`，属性名首字母大写：`setSpeed`
getter 命名	`getXxx`：`getSpeed`
`boolean` 属性的 getter	习惯写成 `isXxx`：`isRunning()`

封装的好处： 数据安全（非法值进不来）、隐藏实现（内部存储方式可以改，调用方式不变）、便于维护（校验逻辑集中在一处）。

3. 用 IDEA 自动生成

光标放在类体内、方法外，按 `Cmd + N`（Windows 为 `Alt + Insert`），打开“生成”菜单：

菜单项	用途
构造函数	勾选字段，生成带参构造方法
Getter / Setter / Getter 和 Setter	生成 `getXxx`、`setXxx`
toString()	打印对象时输出属性值，而不是 `Car@1b6d3586` 这样的地址
equals() 和 hashCode()	比较对象内容是否相等（学集合时用到）
重写方法	学继承时用到

自动生成的 setter 只有 `this.speed = speed;`，没有校验，需要时要自己补上。

四、构造方法

构造方法在 `new` 时自动调用，用来初始化对象。它有三条规则：

1. 名称与类名完全一致，大小写也要相同；
2. 没有返回值类型，连 `void` 也不写；
3. 创建对象时自动调用，不能像普通方法那样用 `对象.方法名()` 调用。

```
public void Car() { }    // 写了 void，就成了一个普通方法，new 时不会被调用
```

1. 无参构造与带参构造

```
public class Car {
    String name;
    String color;
    int speed;

    public Car() { }                                        // 无参构造

    public Car(String name, String color, int speed) {   // 带参构造，与上面构成重载
        this.name = name;
        this.color = color;
        this.speed = speed;
    }
}

Car a = new Car();                          // 调用无参构造
Car b = new Car("小米", "红色", 320);     // 创建时直接设好属性
```

实参的顺序必须与形参一致。`name` 和 `color` 都是 `String`，写反了不会报错，只会把名字和颜色互换。

2. 默认构造方法（必考）

- 类中一个构造方法都不写时，编译器自动提供一个无参构造，所以 `new Car()` 能用。
- 只要自己写了任何一个构造方法，编译器就不再提供。 如果只写了带参构造，`new Car()` 会编译报错：

```
error: constructor Car in class Car cannot be applied to given types;
```

所以写了带参构造后，习惯上把无参构造也手动写上。

3. 用 this(...) 复用构造方法

```
public Car(String color, int speed) {
    this("car", color, speed);    // 调用三参构造，名字默认为 car；必须是第一条语句
}
```

4. 课堂代码：老师的版本

```
public class Test {
    public static void main(String[] args) {
        Car a = new Car("小米", "红色", 320);
        Car b = new Car("宝马M2", "灰色", 300);
        a.fly("白色");
        b.fly("黄色");
    }
}

class Car {
    String color;
    String name;
    int speed;

    public Car(String name, String color, int speed) {
        this.name = name;
        this.color = color;
        this.speed = speed;
    }

    void fly(String color) {
        System.out.printf(this.color + "的" + name + "在" + color + "的云里飞");
    }
}
```

读这段代码要注意：

- 投影上 `new Car(name: "小米", ...)` 中灰色的 `name:`、`color:`、`speed:` 是 IDEA 的参数名提示，不是代码，不要抄。
- `this.color` 是车的颜色，参数 `color` 是云的颜色，所以输出“红色的小米在白色的云里飞”。
- `printf` 不会自动换行，两句输出会连成一行；想分行就改用 `println`，或在末尾加 `\n`。
- `Car` 只写了带参构造，此时 `new Car()` 会编译报错，正好对应上面的“默认构造方法”规则。

5. 课堂代码：Car_Demo2

在自己的 `Car_Demo2` 中加了 `name` 属性，三个构造方法构成重载，两参构造通过 `this(...)` 复用三参构造：

```
public class Car_Demo2 {
    protected String color;
    int speed;
    String name;

    public Car_Demo2() { }
    public Car_Demo2(String color, int speed) {
        this("car", color, speed);
    }
    public Car_Demo2(String name, String color, int speed) {
        this.name = name;
        this.color = color;
        this.speed = speed;
    }
    private void fly() {
        System.out.println(name + " is flying");
```

```
    }
    void fly(String color) {
        System.out.println(this.color + " " + name + " is flying in " + color + " cloud");
    }

    public static void main(String[] args) {
        Car_Demo2 a = new Car_Demo2();
        Car_Demo2 b = new Car_Demo2("red", 180);
        System.out.println(a.color + " " + a.speed);
        System.out.println(b.color + " " + b.speed);
        b.fly();                  // 本类中可以调用 private 方法
        b.fly("black");

        Car_Demo2 c = new Car_Demo2("BMW M2", "grey", 300);
        System.out.println(c.name + " " + c.color + " " + c.speed);
        c.fly("white");
    }
}
```

运行结果：

```
null 0
red 180
car is flying
red car is flying in black cloud
BMW M2 grey 300
grey BMW M2 is flying in white cloud
```

`a` 用无参构造创建，属性都是默认值：`null` 和 `0`。

五、Java 参数传递

Java 只有值传递：调用方法时，把实参的值复制一份给形参。 区别在于这个“值”是什么：基本类型复制的是数据本身，对象和数组复制的是地址。

1. 基本类型：方法内修改不影响外部

```
void main() {
    int a = 10;
    change(a);
    System.out.println(a);      // 10
}
void change(int x) {
    x = 20;                     // 只改了副本 x
}
```

2. 对象类型：通过地址修改对象内容，外部可见

```
void main() {
    User u = new User("Tom");
    change(u);
    System.out.println(u.name); // Jerry
}
void change(User x) {
    x.name = "Jerry";           // x 和 u 指向同一个对象
}
```

3. 数组类型：与对象相同

```
void main() {
    int[] arr = {1, 2, 3};
    change(arr);
    System.out.println(arr[0]); // 100
}
void change(int[] x) {
    x[0] = 100;                 // x 和 arr 指向同一个数组
}
```

4. 易错：在方法内让形参指向新对象

```
void main() {
    User v = new User("Tom");
    replace(v);
    System.out.println(v.name); // Tom
}
void replace(User x) {
    x = new User("Jerry");      // 只改了副本 x 的指向，v 不受影响
}
```

参数类型	复制给形参的值	方法内修改对外部的影响
基本类型	数据本身	无影响
对象	对象地址	修改属性：外部可见；让形参指向新对象：外部不变
数组	数组地址	修改元素：外部可见；让形参指向新数组：外部不变

> 课堂示例写的是 `void main()`，这是 JDK 25 起支持的简化写法（方法可以直接写在类外，入口 `main` 可以不写 `static` 和参数）。考试和旧版 JDK 中，应写成类中的 `public static void main(String[] args)`，`change` 也要加 `static`。

六、课上遇到的问题

现象	原因	解决
`package class13;` 一行报错	`main` 写在了 `Car` 类的大括号外面；JDK 25 把这种文件当作“紧凑源文件”，而它不允许有 `package` 声明	把 `main` 移回类体内
运行 class13 却报 `speed has private access in class12.Car`	IDEA 运行前会构建整个项目，class12 中故意保留的错误语句让构建失败	注释掉 class12 的那一行
方法传不了参	定义时没有声明形参	定义时写出形参，如 `fly(String color)`
生成的 getter/setter 位置不对	按 `Cmd + N` 时光标在方法内部	光标放在类体内、方法外

常见错误

- 只有返回值不同就以为构成重载。
- 构造方法前写了 `void`，它就成了普通方法。
- 只写了带参构造，却仍然调用 `new Car()`。
- 带参构造的实参顺序与形参不一致，同类型的参数被悄悄互换。
- 认为“对象是引用传递”。Java 只有值传递，传的是地址的副本。
- 在方法里写 `x = new User(...)`，以为能改变外部变量的指向。
- 在其他类中直接访问 `private` 属性。

本课小结

- 重载看参数，不看返回值。
- `private` 只限本类；默认和 `protected` 在同包内都可访问；`protected` 还允许不同包的子类通过继承访问。
- 封装 = `private` 属性 + 公共的 getter/setter，在 setter 中校验数据。
- 构造方法与类同名、没有返回值类型、`new` 时自动调用；自己写了构造方法后，编译器不再提供无参构造。
- Java 只有值传递：基本类型传数据，对象和数组传地址。

自测题

1. `int add(int a, int b)` 与 `double add(int a, int b)` 能否同时存在？为什么？
2. 类中只写了 `Car(String name)`，执行 `new Car()` 会怎样？如何解决？
3. `public void Car() { }` 是构造方法吗？
4. 为什么在 setter 中可以拦住非法数据，而直接给属性赋值不行？
5. 写出下面代码的输出：`int[] arr = {1, 2, 3}; change(arr); System.out.println(arr[0]);`，其中 `change` 执行 `x[0] = 100;`。
6. 方法 `replace(User x)` 中执行 `x = new User("Jerry");`，调用后外部对象的 `name` 是否改变？
7. 不同包的子类中，`this.color` 和 `new Car().color` 哪个能访问 `protected` 的 `color`？